



KUBERNETES CORE COMPONENTS

Everything you need to build, run
and scale modern applications



ARCHITECTURE



CHEAT SHEET



[linkedin.com/in/siranjeevi1](https://www.linkedin.com/in/siranjeevi1)



siranjeevis.netlify.app



PODS

Smallest Deployable Units in Kubernetes

A Pod is a group of one or more containers that share storage, network, and a specification for how to run containers.



KEY CHARACTERISTICS



One or more containers
Tightly coupled containers that work together.



Shared Resources
Share the same network namespace, IP address, and storage volumes.

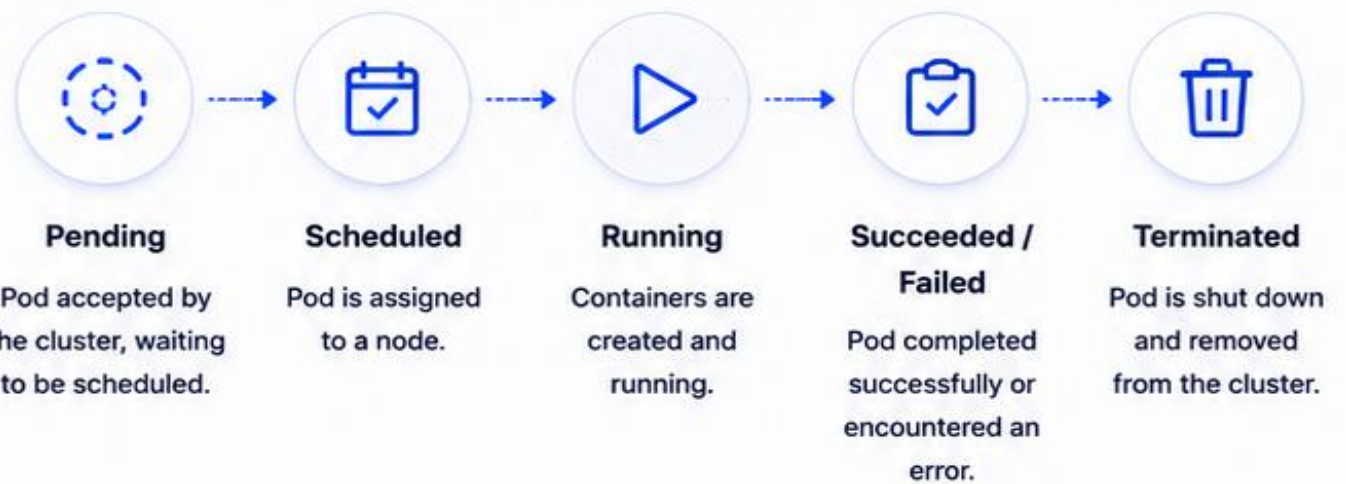


Ephemeral
Pods are not meant to live forever. They can be created, destroyed, and recreated.



Controlled by higher-level controllers
Such as Deployments, StatefulSets, DaemonSets, Jobs.

POD LIFECYCLE



MULTI-CONTAINER POD

Containers in a pod are designed to work closely together.



Share same IP and network



Share storage volumes



Communicate via localhost

COMMON USE CASES



Web application + log shipper



App + caching sidecar



Service mesh (sidecar proxy)



Monitoring agent + exporter



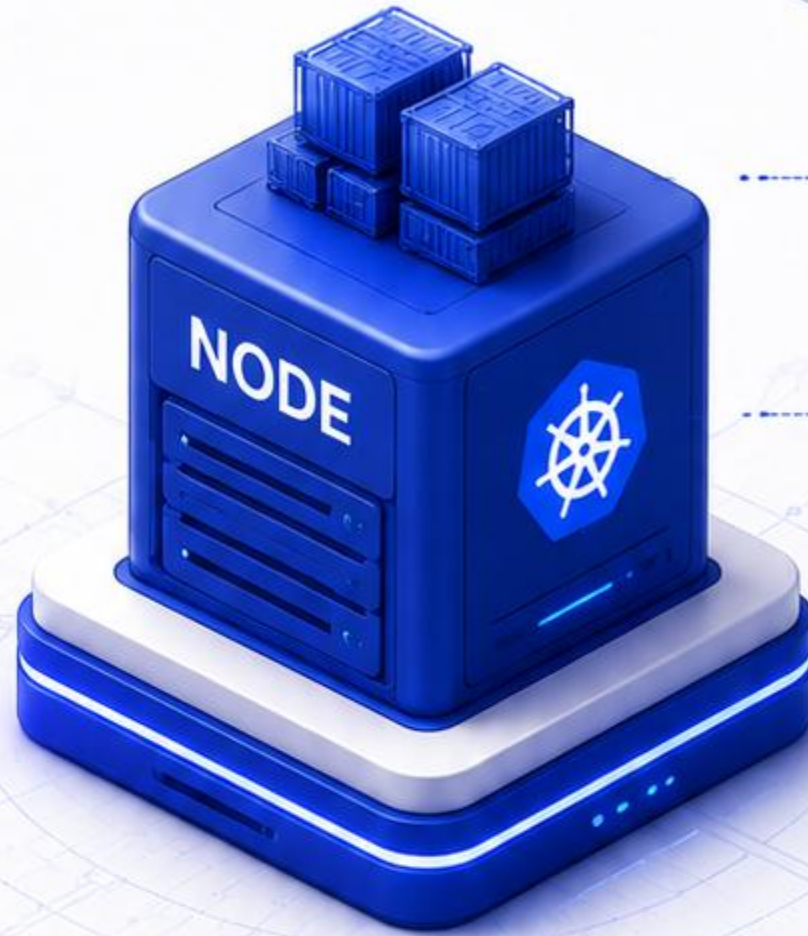
Init container for setup tasks



NODES

The Workhorses of Kubernetes

A Node is a worker machine in Kubernetes, either a virtual or physical machine, where your containers (Pods) are deployed and run.



Runs Pods

Provides Resources

Monitored by Control Plane

WHAT IS A NODE?



- ✓ A node is a machine (VM or physical) in the Kubernetes cluster.
- ✓ Nodes run containerized applications (Pods).
- ✓ Managed by the control plane and communicate via **kubelet**.

NODE TYPES



CONTROL PLANE NODE

- Runs control plane components
- Manages the cluster
- Scheduling & decision making
- Can also run workloads (optional)



WORKER NODE

- Runs application workloads (Pods)
- Contains container runtime
- Managed by control plane
- Does not manage the cluster

KEY COMPONENTS



kubelet

Agent that runs on each node and ensures containers are running in a Pod.



kube-proxy

Maintains network rules on the node and enables service connectivity.



Container Runtime

Software responsible for running containers (containerd, Docker, CRI-O).



OS & System Services

Underlying OS, networking, storage and other system services.

NODE RESPONSIBILITIES



Run Pods

Executes and keeps Pods running.



Provide Resources

Offers CPU, memory, storage and network to Pods.



Health Monitoring

Reports node health status to control plane.



Log & Metrics

Sends logs and metrics for monitoring.



Networking

Handles Pod network interfaces and routing.



Storage

Mounts volumes for Pods from various sources.



Security

Enforces security policies and isolates workloads.



Updates & Patches

Node OS and components are updated regularly.



DEPLOYMENTS

Declarative Updates for Your Applications

Deployments manage ReplicaSets and ensure the desired number of Pod replicas are running. They handle rollouts, rollbacks and scaling with ease.



HOW DEPLOYMENTS WORK



1. Define Desired State
You define the desired state in a Deployment (e.g., 3 replicas of Nginx).



2. Creates ReplicaSet
Deployment creates a ReplicaSet to achieve the desired state.



3. Pods are Created
ReplicaSet creates and maintains the required number of Pods.



4. Self-Healing
If a Pod fails, it is replaced automatically to maintain the state.



5. Updates & Rollbacks
Deployment updates Pods with zero downtime and can rollback if needed.

KEY FEATURES



Declarative
Define the desired state, Deployment makes it happen.



Rolling Updates
Update Pods gradually with zero downtime.



Rollbacks
Easily rollback to a previous revision.

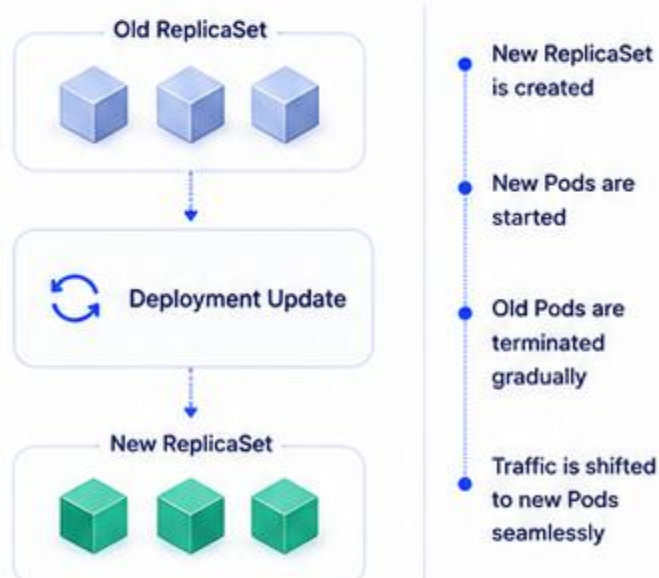


Scaling
Scale your application up or down.



Self-Healing
Automatically replace failed Pods.

ROLLING UPDATE FLOW



YAML EXAMPLE

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
          ports:
            - containerPort: 80
```

COMMON USE CASES



Stateless application deployments



Blue/Green deployments



Canary deployments



Auto scaling applications



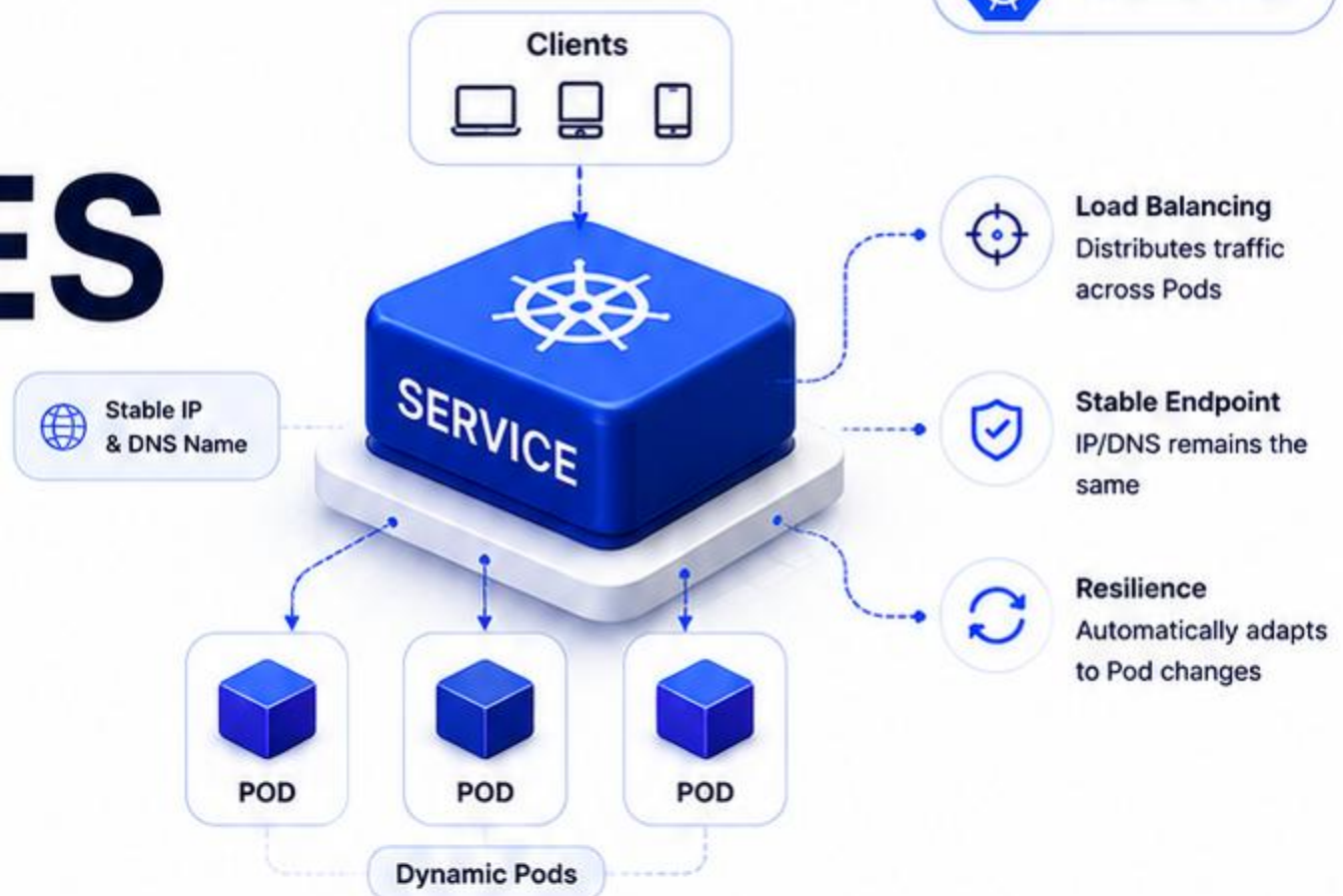
Zero downtime updates



SERVICES

Stable Networking for Your Applications

A Service is an abstraction that defines a logical set of Pods and a policy by which to access them. It provides a stable IP and DNS name that never changes, even if Pods come and go.



SERVICE TYPES



ClusterIP (Default)

Exposes the Service on a cluster-internal IP. Accessible only inside the cluster.



NodePort

Exposes the Service on each Node's IP at a static port (30000-32767). Accessible from outside the cluster.



LoadBalancer

Exposes the Service externally using a cloud provider's load balancer. Accessible from outside the cluster.



ExternalName

Maps the Service to an external DNS name. No proxying, just a CNAME record.

HOW SERVICE WORKS



TRAFFIC FLOW EXAMPLE (CLUSTERIP)



💡 Even if Pods are created, destroyed or rescheduled, the Service IP and DNS remain the same.

KEY FEATURES



Stable Access
Consistent IP/DNS for your app.



Abstraction
Decouples clients from Pod lifecycle.



Load Balancing
Distributes traffic evenly.



Support Multiple Ports
Expose one or more ports.



Self-Healing
Automatically adjusts to Pod changes.



Works with DNS
Easy service discovery inside the cluster.

COMMON USE CASES



E-commerce
Frontend to backend communication



Microservices
Inter-service communication



APIs
Expose APIs to internal or external users



Web Applications
Expose web apps via LoadBalancer



Monitoring
Expose monitoring dashboards



Message Brokers
Expose message brokers



Databases
Expose databases internally



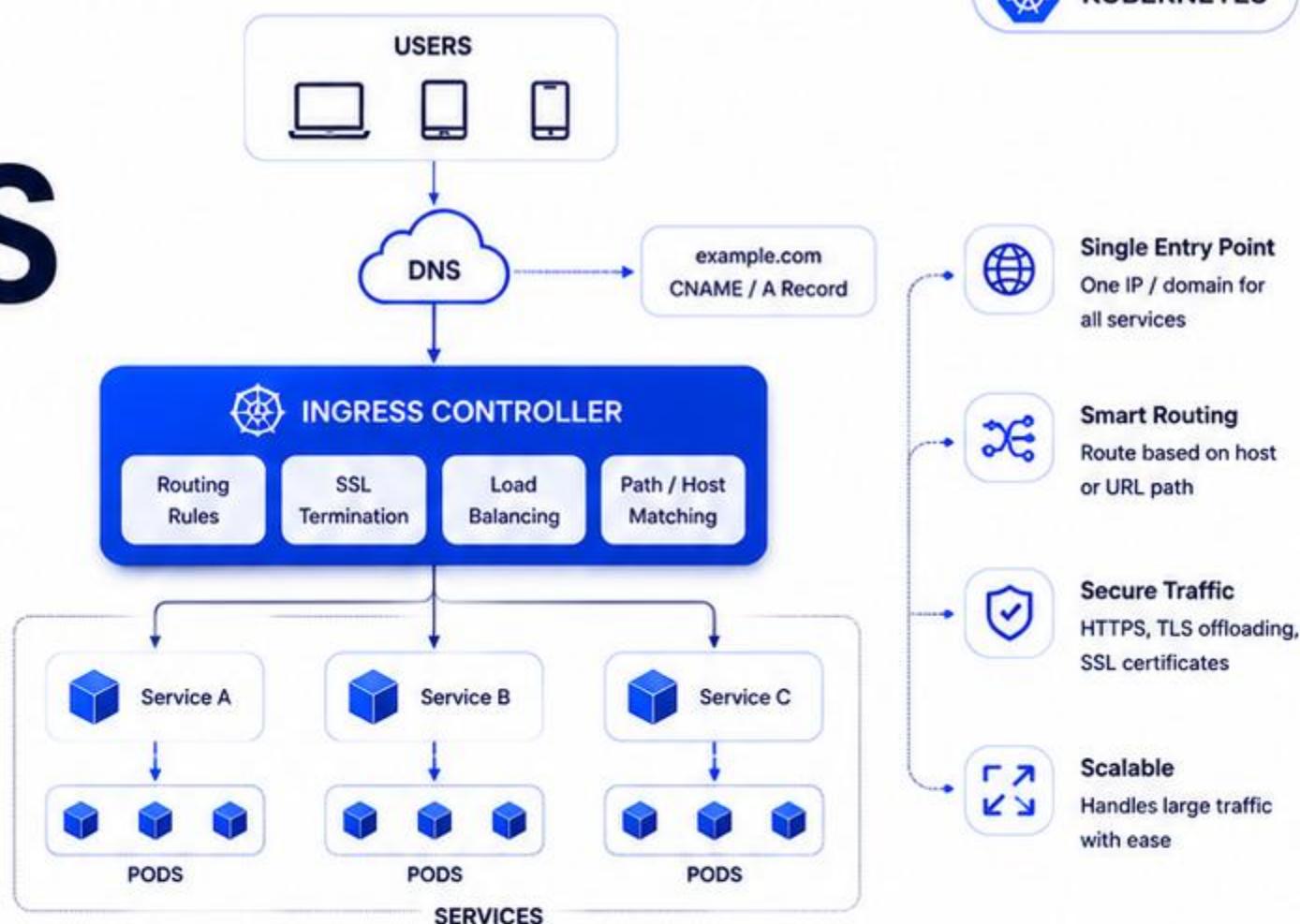
CI/CD Tools
Expose CI/CD services



INGRESS

Smart HTTP/HTTPS Routing for Your Services

Ingress manages external access to services in a cluster, typically HTTP and HTTPS. It provides load balancing, SSL termination, and name-based or path-based routing using a single entry point.



INGRESS RULES EXAMPLE

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: example-ingress
spec:
  rules:
  - host: example.com
    http:
      paths:
      - path: /app1
        pathType: Prefix
        backend:
          service:
            name: service-a
            port:
              number: 80
      - path: /app2
        pathType: Prefix
        backend:
          service:
            name: service-b
            port:
              number: 80
```



Ingress requires an Ingress Controller (NGINX, Traefik, HAProxy, etc.) to work.

HOW INGRESS WORKS



HOST BASED vs PATH BASED ROUTING



You can combine both host-based and path-based rules in a single Ingress.

KEY FEATURES

- External Access:** Expose Services outside the cluster
- Load Balancing:** Distribute traffic across multiple Pods
- SSL Termination:** Offload TLS/HTTPS at the Ingress level
- TLS Support:** Use TLS certificates for secure communication
- Advanced Routing:** Route based on host, path, headers, and more
- Controller Integration:** Works with NGINX, Traefik, HAProxy and more

COMMON USE CASES

- Microservices:** Expose multiple services via a single domain
- Multi-Tenant:** Provide isolated access for multiple teams/clients
- Path-Based Apps:** Route different parts of an app to different services
- SSL Offloading:** Improve performance by offloading SSL at Ingress
- Blue/Green Deployments:** Route traffic to new or old versions
- Canary Releases:** Gradually roll out changes to a subset of users
- API Gateways:** Expose APIs securely with routing & auth rules
- Web Applications:** Serve web apps and static content efficiently

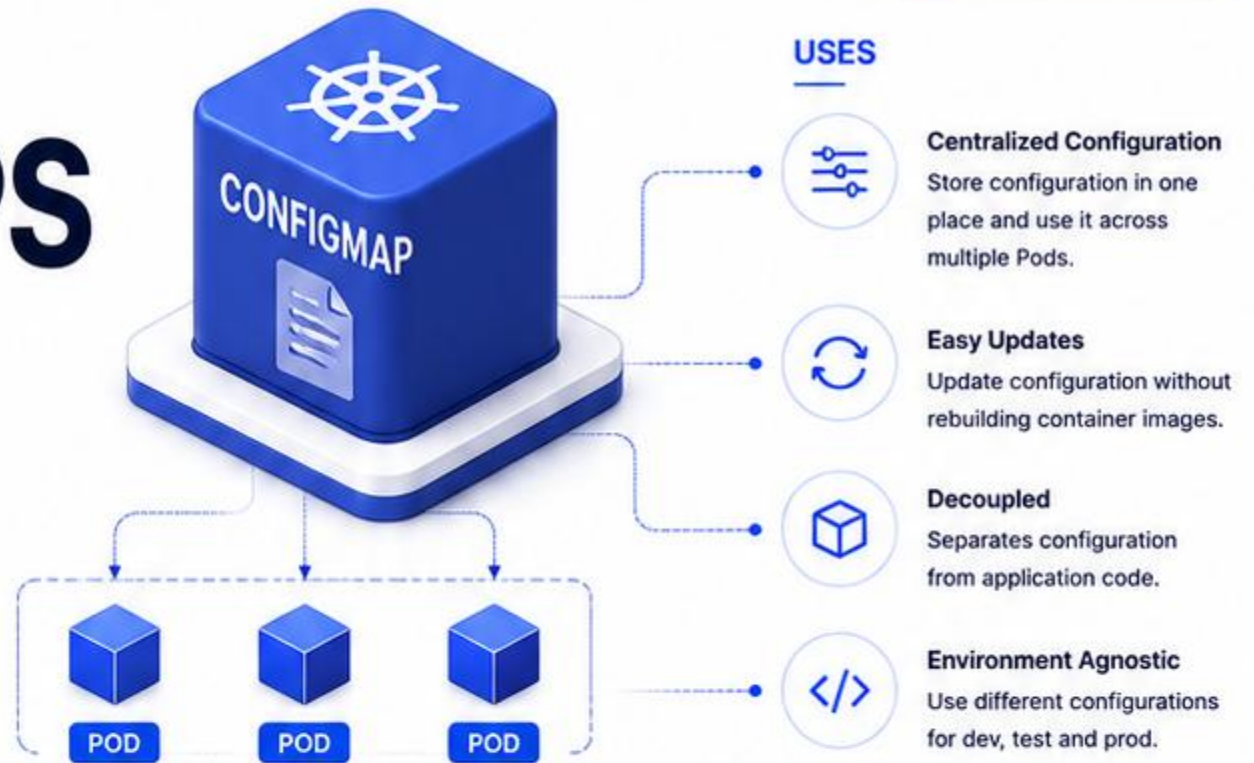




CONFIGMAPS

Store Non-Sensitive Configuration Data

ConfigMaps allow you to decouple configuration from container images and keep your applications flexible and portable.



WHAT IS A CONFIGMAP?



ConfigMap is an API object used to store non-confidential data in key-value pairs.

Ideal for:

- ✓ Application configuration files
- ✓ Environment variables
- ✓ Command-line arguments
- ✓ Feature flags
- ✓ Log levels, connection strings (non-secret)

HOW CONFIGMAP WORKS



1. Create
Define key-value pairs in a ConfigMap.



2. Mount / Use
Mount as files in a volume or use as environment variables.



3. Consume
Pods consume the configuration at runtime.



4. Update
Update the ConfigMap and Pods get new config (on restart or with reload).

WAYS TO USE IN PODS



As Volume (Files)
Mount ConfigMap as a volume to read configuration files inside the container.



As Environment Variables
Expose ConfigMap values as environment variables in the container.

EXAMPLE CONFIGMAP (YAML)

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
  namespace: default
data:
  app.properties: |
    app.name=MyApp
    app.env=production
    log.level=info
  db.host: "postgres.default.svc.cluster.local"
  feature.flag.newUI: "true"
```

Use as Volume



ConfigMap
app-config



Mounted as files
in /config



Pod
(reads files)

Files in
Container
/config

- app.properties
- db.host
- feature.flag.newUI

Use as Environment Variables



ConfigMap
app-config



Injected as
Environment
Variables



Pod
(uses env vars)

Environment Variables
in Container

- DB_HOST
- APP_ENV
- LOG_LEVEL
- FEATURE_FLAG_NEWUI

KEY FEATURES



Store non-sensitive configuration data



Decouple config from container image



Update config without rebuilding images



Use as files or environment variables



Namespace scoped



Version control friendly

COMMON USE CASES



Application Configuration



Database Settings



Feature Flags



Command-line Arguments



Nginx / Apache Configs



Log Levels & Formats



Environment Specific Configs



Third-party Service Endpoints





SECRETS

Secure Sensitive Data in Your Cluster

Secrets let you store and manage sensitive information such as passwords, tokens, keys, and certificates. They are encrypted in etcd and can be mounted into Pods securely.



WHY USE SECRETS?



Secure
Sensitive data is encrypted in etcd.



Access Controlled
RBAC ensures only authorized access.



Decoupled
Keep secrets separate from application code.



Easy Rotation
Update secrets without rebuilding images.

WHAT IS A SECRET?



A Secret is an object that contains small amounts of sensitive data in key-value pairs.

Ideal for storing:

- ✓ Passwords & Tokens
- ✓ API Keys & OAuth Credentials
- ✓ TLS Certificates
- ✓ SSH Keys
- ✓ Other Sensitive Information

HOW SECRETS WORK



1. Create

Create a Secret with sensitive data.



2. Store

Kubernetes stores it encrypted in etcd.



3. Mount

Secret is mounted into Pods as files or environment variables.



4. Use

Applications use the data at runtime.

SECRET TYPES



Opaque

General purpose key-value pairs.



kubernetes.io/dockerconfigjson

Store Docker registry credentials.



kubernetes.io/tls

Store TLS certificate and private key.



Create custom secrets for your specific needs.

EXAMPLE: CREATE SECRET (YAML)

```
apiVersion: v1
kind: Secret
metadata:
  name: my-secret
  type: Opaque
data:
  username: YWRtaW4= # base64 encoded "admin"
  password: UGFzc3dvcmQxMjM= # base64 encoded "Password123"
```



Data is base64 encoded. It is not encrypted. Kubernetes encrypts Secrets at rest in etcd (if enabled).

HOW TO CONSUME SECRETS



As Environment Variables

```
env:
- name: DB_PASSWORD
  valueFrom:
    secretKeyRef:
      name: my-secret
      key: password
```



As Volume (Files)

```
volumeMounts:
- name: secret-volume
  mountPath: /etc/secret
  readOnly: true
```



Secret
(my-secret)



Pod
(Consumes Secret)

KEY FEATURES



Encrypted in etcd (at rest)



RBAC protected access



Can be updated and rotated



Decouples config from code



Mount as files or env vars



Namespace scoped

COMMON USE CASES



Database Credentials



API Keys & Tokens



TLS / SSL Certificates



Docker Registry Credentials



OAuth Client Secrets



SSH Private Keys



Third-party Service Credentials



License Keys

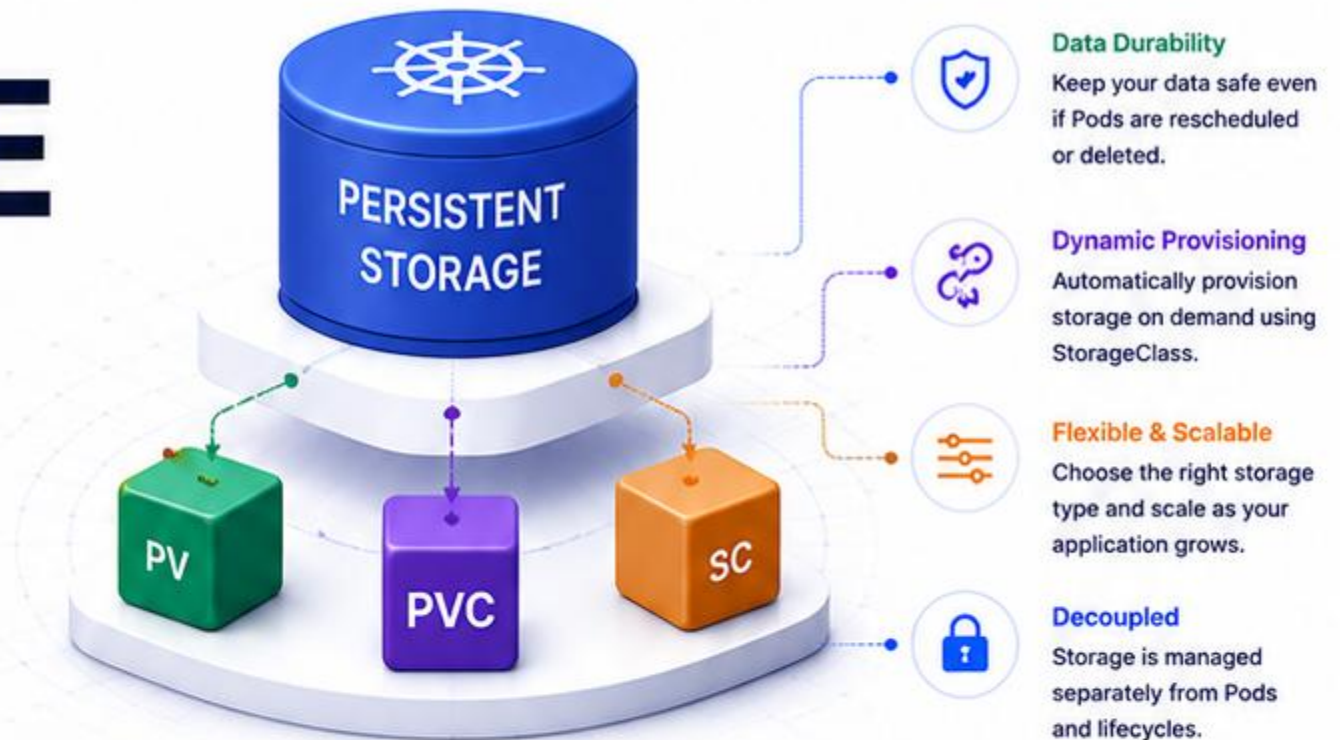




STORAGE

Persistent Storage for Stateful Workloads

Kubernetes Storage allows you to persist data beyond the lifecycle of Pods. It is built on three key concepts: **PV**, **PVC** and **StorageClass**.



PERSISTENT VOLUME (PV)



A PV is a piece of storage in the cluster that has been provisioned by an admin or dynamically provisioned using a StorageClass.

- ✓ Cluster resource
- ✓ Independent of any Pod
- ✓ Has a lifecycle independent of Pods
- ✓ Can be backed by various storage systems (NFS, EBS, Ceph, etc.)

PERSISTENT VOLUME CLAIM (PVC)



A PVC is a request for storage by a user. It is similar to a Pod. Pods consume storage by using PVC.

- ✓ Namespace scoped
- ✓ Requests specific storage size and access mode
- ✓ Binds to a suitable PV
- ✓ Used by Pods as a volume

STORAGE CLASS (SC)



A StorageClass provides a way for admins to describe the "classes" of storage they offer (e.g., SSD, HDD, high IOPS, etc.).

- ✓ Defines provisioner, parameters, and policies
- ✓ Enables dynamic provisioning of PVs
- ✓ PVC references a StorageClass to get storage on demand

HOW STORAGE WORKS



1. PVC Created

User creates a PVC requesting storage.

2. StorageClass Used

PVC references a StorageClass.

3. PV Provisioned

StorageClass provisions a suitable PV (dynamic) or uses existing PV (static).

4. Bound

PVC is bound to the PV.

5. Pod Uses PVC

Pod mounts the PVC and starts using the storage.

EXAMPLE: PVC YAML

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
  storageClassName: fast-ssd
```

KEY FEATURES

- Data Persistence**
Data survives Pod restarts, rescheduling and failures.
- Dynamic Provisioning**
Automatically create storage when PVC is created.
- Multiple Access Modes**
Supports RWO, ROX, RWX access modes.
- Pluggable Storage**
Works with many storage backends.
- Quota & Isolation**
Namespaces can have their own storage limits.
- Scalability**
Scale storage based on application needs.

COMMON USE CASES

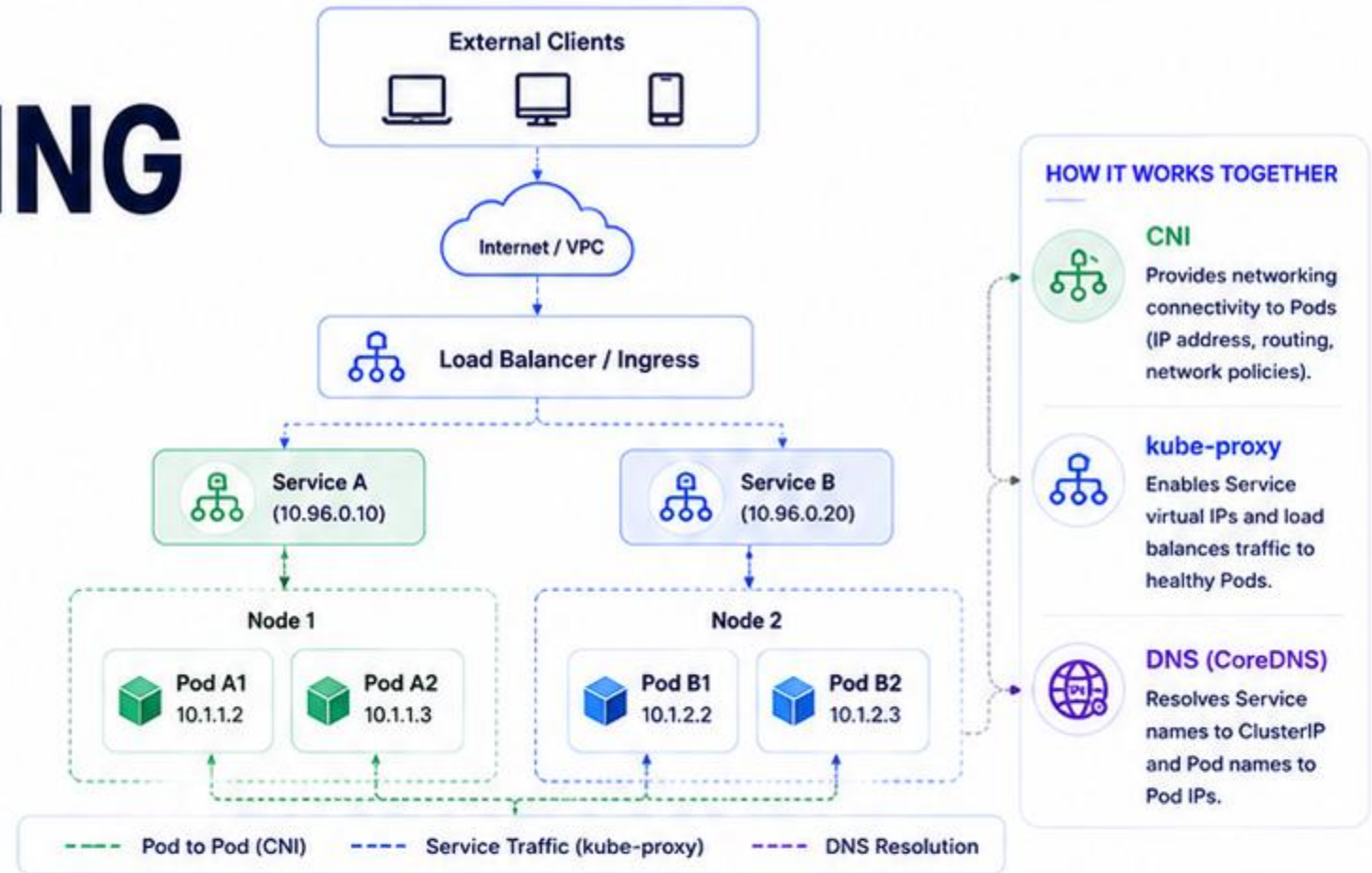
- Databases**
MySQL, PostgreSQL, MongoDB
- Analytics**
Elasticsearch, ClickHouse
- File Storage**
Shared files, media and documents
- Backup & Restore**
Storing backups, snapshots
- Logging**
Loki, Fluentd storage
- Caching**
Redis, Memcached persistence
- AI/ML Workloads**
Model data, training datasets
- CI/CD Artifacts**
Build artifacts, reports



NETWORKING

CNI, kube-proxy & DNS Overview

Kubernetes networking enables Pods, Services and external users to communicate securely and reliably across the cluster.



CNI (CONTAINER NETWORK INTERFACE)



CNI plugins are responsible for setting up network interfaces in Pods and connecting them across nodes.

Responsibilities

- ✓ Assign IP address to Pods
- ✓ Enable Pod to Pod communication
- ✓ Apply Network Policies
- ✓ Support multiple network plugins

Popular CNI Plugins



KUBE-PROXY



kube-proxy runs on each node and maintains network rules to enable Service abstraction and load balancing.

Responsibilities

- ✓ Maintain iptables/IPVS rules
- ✓ Forward traffic to the right Pods
- ✓ Support ClusterIP, NodePort, LoadBalancer
- ✓ Health check and update endpoints

Modes



DNS (COREDNS)



CoreDNS is the DNS server in Kubernetes that handles name resolution for Services and Pods.

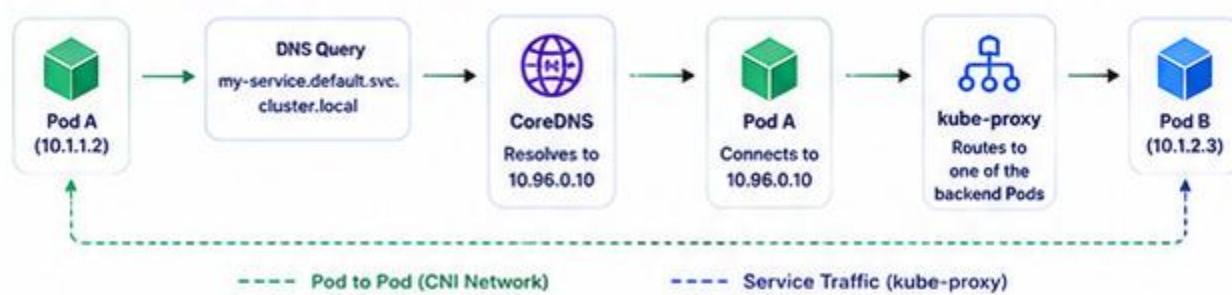
Responsibilities

- ✓ Resolve Service names to ClusterIP
- ✓ Resolve Pod names to Pod IPs
- ✓ Support DNS records and domains
- ✓ Configurable via ConfigMaps

Key Records



NETWORK TRAFFIC FLOW EXAMPLE



SERVICE TYPES & TRAFFIC

Service Type	Exposed As	Traffic Flow
ClusterIP	Internal IP (10.x.x.x)	Within the cluster only
NodePort	Node IP : <NodePort>	External access via node IP and port
LoadBalancer	Cloud LB IP	External access via Cloud Load Balancer
ExternalName	DNS CNAME	Maps to external DNS name (not a proxy)

KEY POINTS TO REMEMBER

- CNI provides connectivity between Pods across nodes.
- Network Policies control how Pods communicate with each other.
- kube-proxy makes Services work by forwarding traffic to the right Pods.
- All three components work together to enable secure, reliable communication.
- DNS is the backbone for name resolution in Kubernetes.
- Monitor networking with tools like Prometheus, Grafana & Cilium Hubble.

COMMON USE CASES

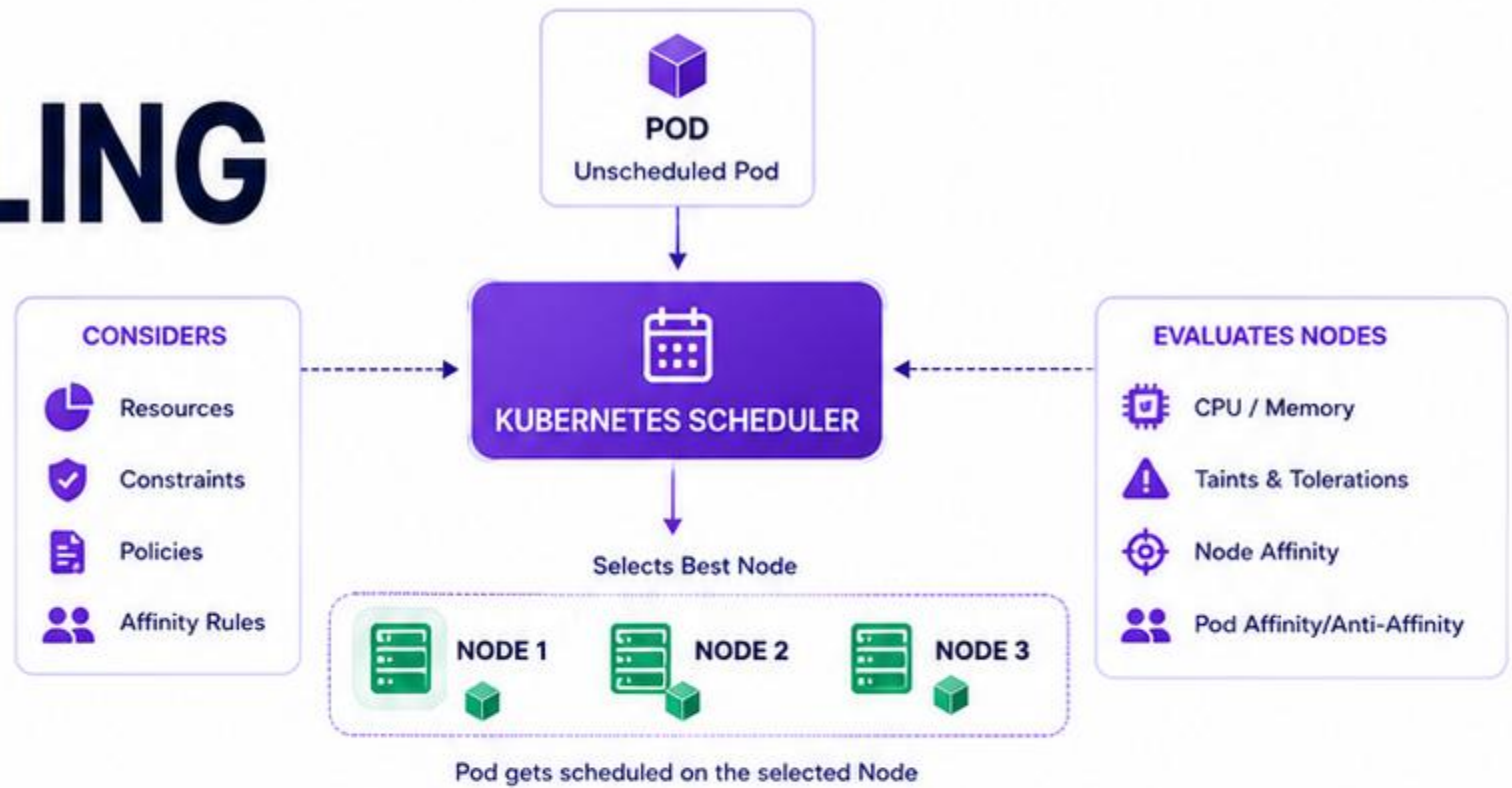
- Microservices Communication
- Multi-Tenant Environments
- Internal API Access
- Blue/Green Deployments
- Monitoring & Logging
- Service Discovery
- Network Policy Enforcement
- Hybrid / Multi-Cloud Networking



SCHEDULING

Scheduler, Taints, Tolerations, Affinity

Kubernetes Scheduler places Pods on the most suitable Nodes based on resources, constraints and policies.



SCHEDULER



The Scheduler watches for unscheduled Pods and assigns them to the most suitable Node.

Key Responsibilities

- ✓ Filter nodes that can't run the Pod
- ✓ Score nodes based on policies
- ✓ Bind Pod to the best fit Node

TAINTS & TOLERATIONS



Taints repel Pods from Nodes. Tolerations allow Pods to be scheduled on tainted Nodes.

Key Points

- ✓ Taint is applied on a Node
- ✓ Tolerations are set on a Pod
- ✓ Pod can tolerate matching taints

NODE AFFINITY



Node Affinity allows you to schedule Pods on nodes that match specific labels.

Key Points

- ✓ Hard: `requiredDuringScheduling IgnoredDuringExecution`
- ✓ Soft: `preferredDuringScheduling IgnoredDuringExecution`

POD AFFINITY / ANTI-AFFINITY



Pod Affinity/Anti-Affinity lets you schedule Pods based on the presence (or absence) of other Pods.

Key Points

- ✓ Keep related Pods close (affinity)
- ✓ Keep conflicting Pods apart (anti-affinity)

SCHEDULING FLOW



QUICK REFERENCE

Scheduler	Places Pods on the most suitable Nodes.
Taint	Applied on Node to repel Pods.
Toleration	Applied on Pod to tolerate taints.
Node Affinity	Schedule Pod on nodes with specific labels.
Pod Affinity	Schedule Pod near other matching Pods.
Pod Anti-Affinity	Schedule Pod away from conflicting Pods.

KEY TAKEAWAYS

- ✓ Scheduler ensures optimal placement of Pods.
- ✓ Use Taints & Tolerations to control which Pods can run on a Node.
- ✓ Use Node Affinity to target Pods to specific Nodes.
- ✓ Use Pod Affinity / Anti-Affinity to manage Pod placement relative to others.
- ✓ Together, these features help build efficient, resilient and reliable workloads.



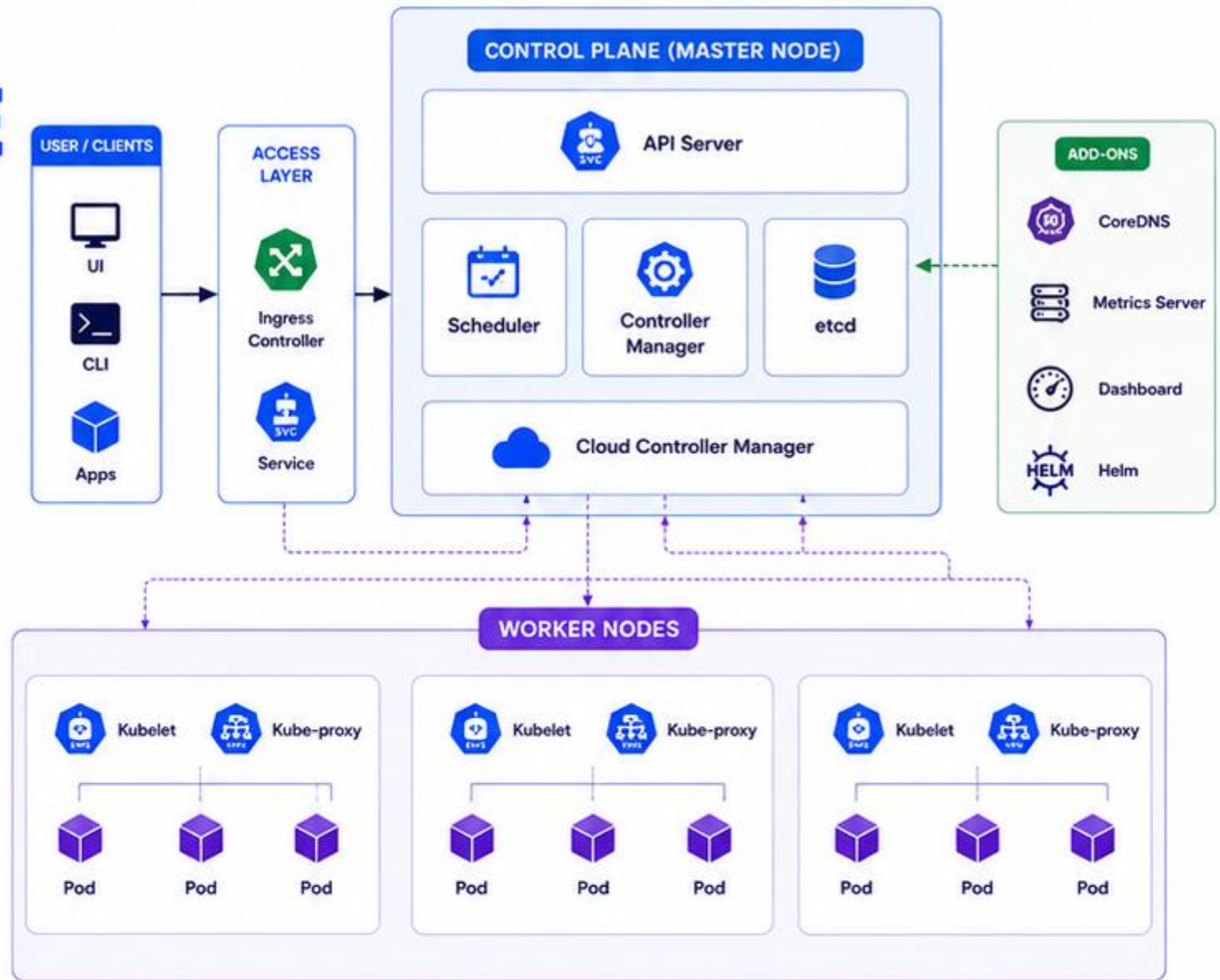
KUBERNETES ARCHITECTURE + CHEAT SHEET



Kubernetes is a container orchestration platform that automates deployment, scaling, and management of containerized applications.

KEY COMPONENTS

	API Server	Exposes the Kubernetes API.
	Scheduler	Assigns Pods to the best Nodes.
	Controller Manager	Runs controllers (e.g., Replication, Node, Endpoint).
	etcd	Consistent and highly-available key-value store.
	Kubelet	Ensures containers are running in a Pod.
	Kube-proxy	Maintains network rules for Services.



POD COMMANDS

<code>kubectl get pods</code>	List all pods
<code>kubectl get pods -o wide</code>	List pods with more details
<code>kubectl describe pod <pod-name></code>	Show pod details
<code>kubectl logs <pod-name></code>	Show pod logs
<code>kubectl exec -it <pod-name> -- /bin/sh</code>	Access pod shell
<code>kubectl delete pod <pod-name></code>	Delete a pod

DEPLOYMENT COMMANDS

<code>kubectl get deploy</code>	List deployments
<code>kubectl create deploy <name> --image=<image></code>	Create deployment
<code>kubectl describe deploy <name></code>	Describe deployment
<code>kubectl scale deploy <name> --replicas=3</code>	Scale deployment
<code>kubectl rollout status deploy <name></code>	Check rollout status
<code>kubectl rollout undo deploy <name></code>	Rollback deployment

SERVICE COMMANDS

<code>kubectl get svc</code>	List services
<code>kubectl expose deploy <name> --type=ClusterIP --port=80</code>	Expose deployment
<code>kubectl describe svc <svc-name></code>	Describe service
<code>kubectl get endpoints <svc-name></code>	Show service endpoints
<code>kubectl delete svc <svc-name></code>	Delete service

NAMESPACE & CONTEXT

<code>kubectl get ns</code>	List namespaces
<code>kubectl create ns <name></code>	Create namespace
<code>kubectl config get-contexts</code>	List contexts
<code>kubectl config use-context <name></code>	Switch context
<code>kubectl config current-context</code>	Show current context

NODE COMMANDS

<code>kubectl get nodes</code>	List nodes
<code>kubectl get nodes -o wide</code>	List nodes with details
<code>kubectl describe node <node-name></code>	Describe node
<code>kubectl drain <node-name> --ignore-daemonsets</code>	Drain node
<code>kubectl uncordon <node-name></code>	Uncordon node

USEFUL COMMANDS

<code>kubectl get all</code>	Get all resources
<code>kubectl get all -A</code>	Get all in all namespaces
<code>kubectl describe all</code>	Describe all resources
<code>kubectl apply -f <file.yaml></code>	Apply configuration
<code>kubectl delete -f <file.yaml></code>	Delete configuration
<code>kubectl top nodes</code>	Show node resource usage
<code>kubectl top pods</code>	Show pod resource usage

KUBERNETES RESOURCE TYPES

 Pod Smallest deployable unit in Kubernetes.	 Deployment Manages replica sets and pod updates.	 ReplicaSet Ensures specified number of pod replicas.	 StatefulSet Manages stateful applications.	 DemonSet Runs a pod on all (or selected) nodes.	 Service Exposes pods as a network service.	 Ingress Manages external access to services.	 ConfigMap Stores non-sensitive configuration.	 Secret Stores sensitive data.	 PersistentVolumeClaim Requests storage for a pod.
---	--	--	--	---	--	--	---	---	---

